

DBMS LAB -10

Neo4j

NAME: Dishan D

SRN: PES1UG23CS196

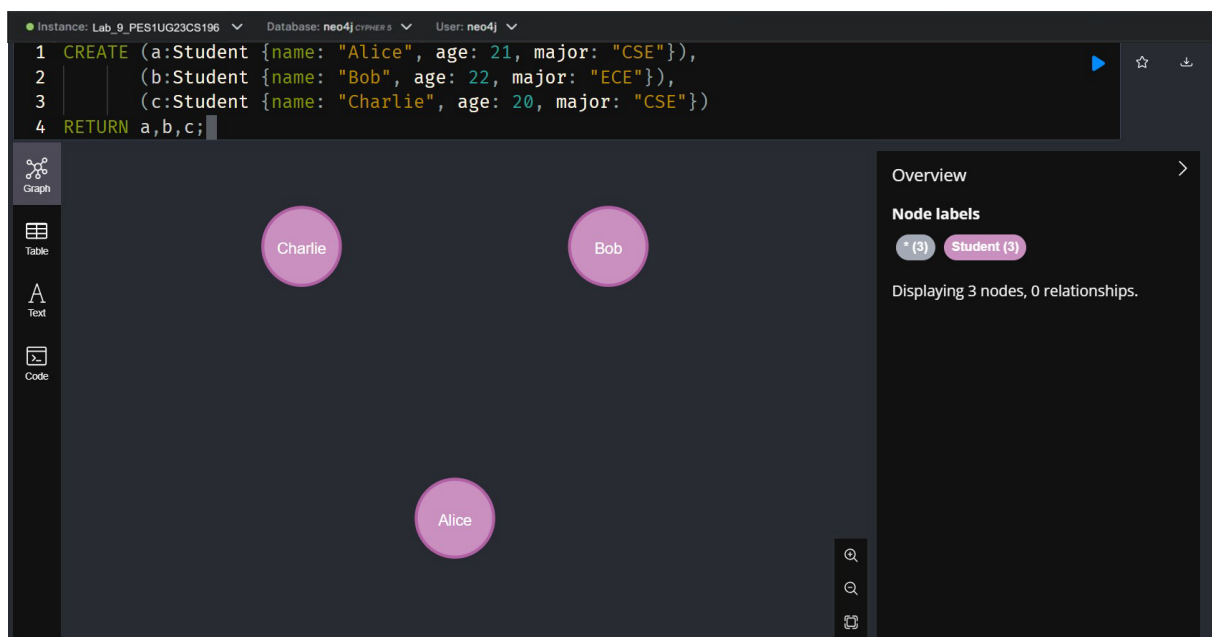
SEM 5

SEC 'D'

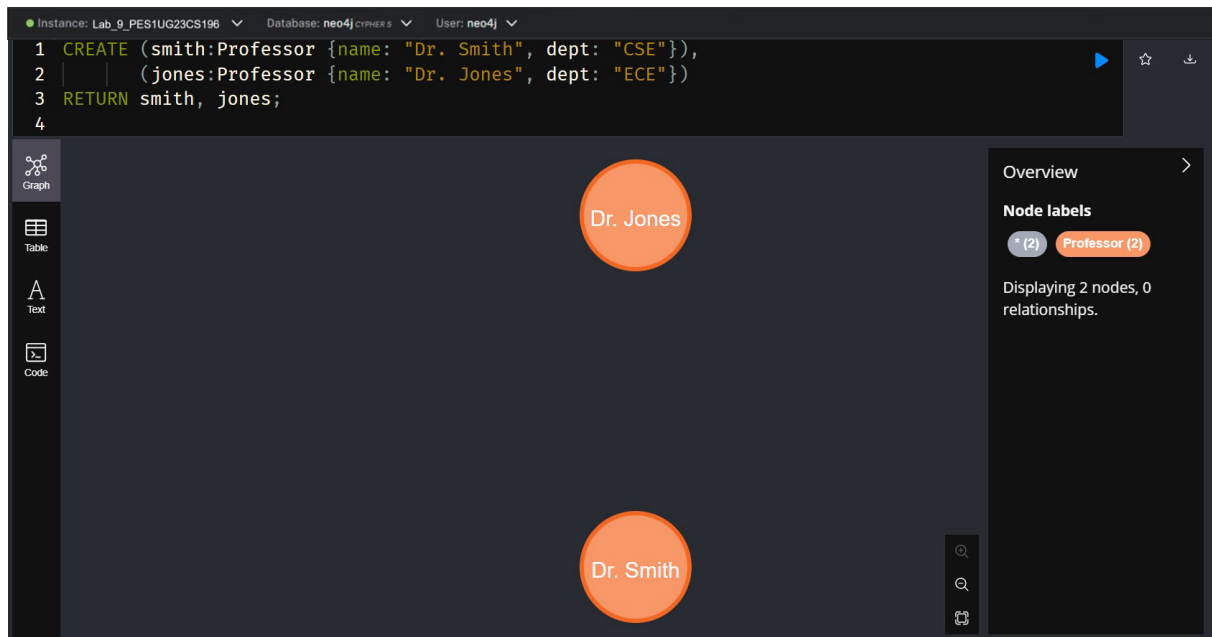
DATE:7/11/2025

Part A: Create Nodes

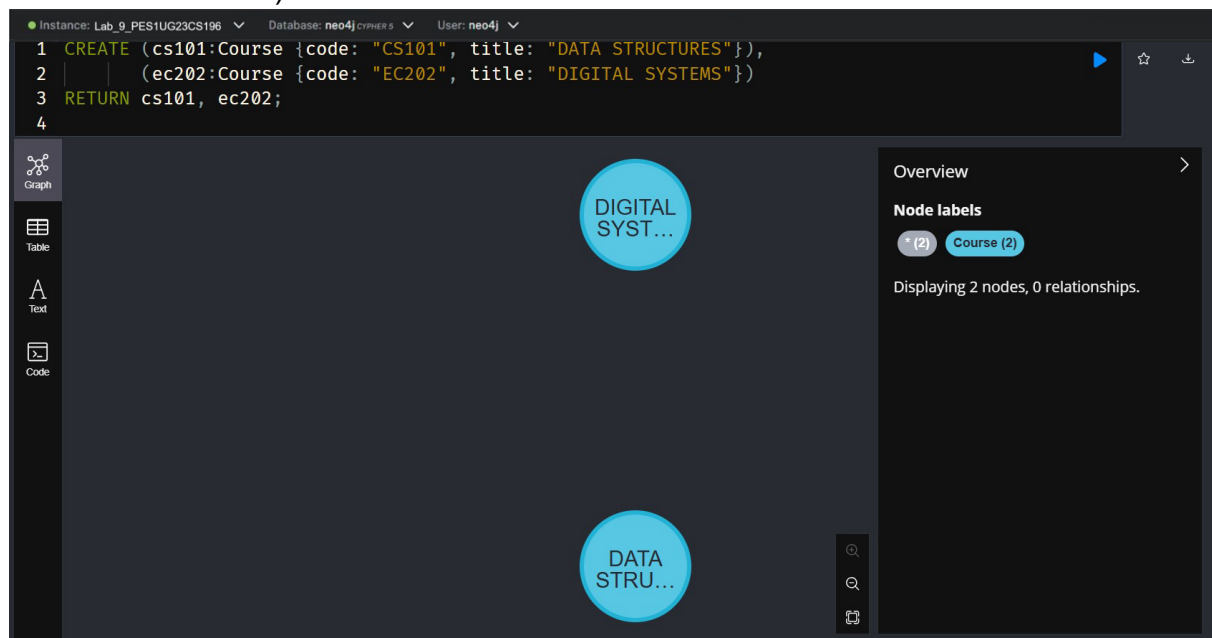
1. Creation of Student Nodes with name, age and major. We have Alice, 21, CSE & Bob, 22, ECE and Charlie, 20, CSE.



2. Creation of Professor Nodes, Dr. Smith for the CSE department and Dr. Jones for the ECE department

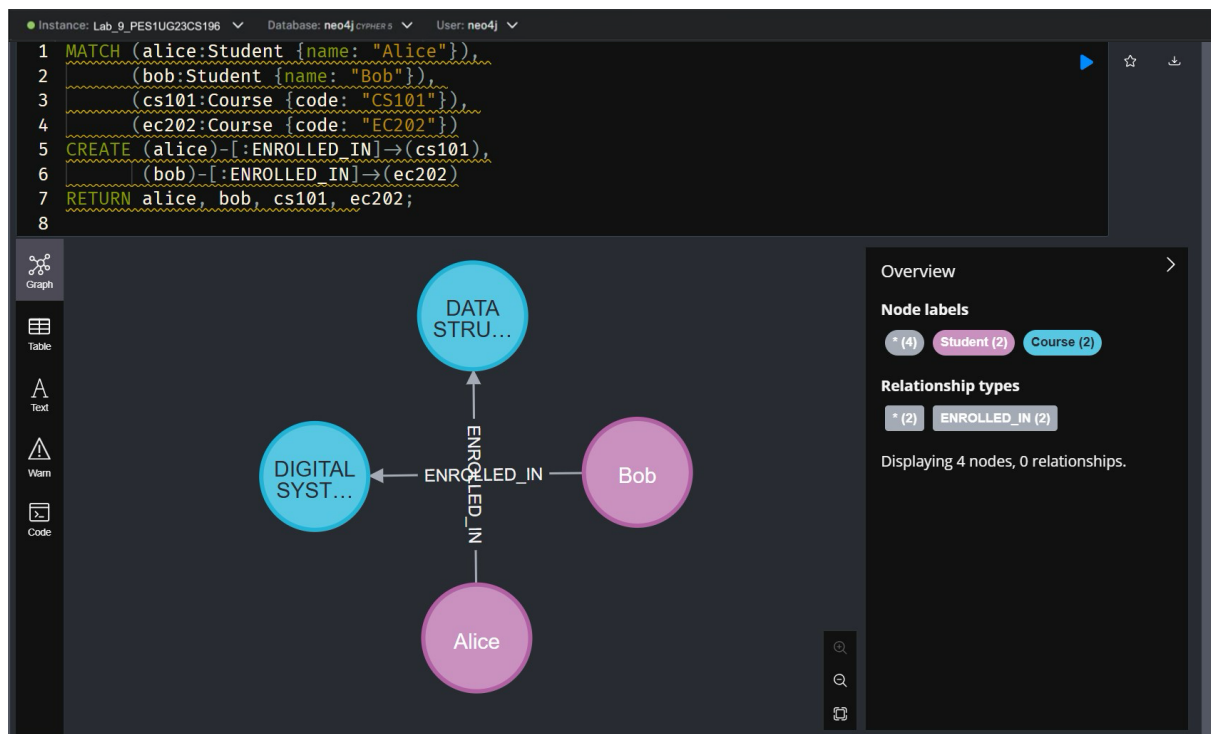


3. Creation of Course Nodes (CODE: CS101, DATA STRUCTURES & CODE: EC202, DIGITAL SYSTEMS)

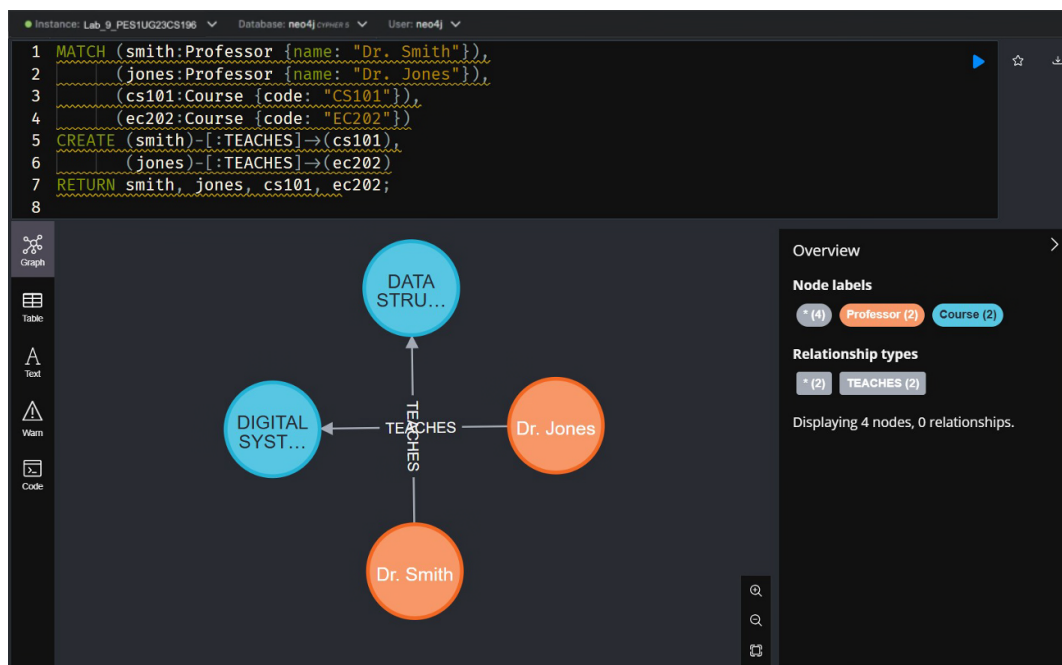


Part B: Create Relationships

4. Show that Alice has enrolled into "CS101" and Bob has enrolled into "EC202"



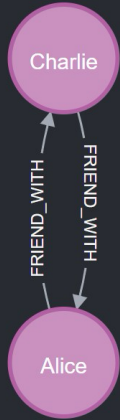
5. Showing that professor Dr. Smith teaches “CS101” and Dr. Jones teaches “EC202”



6. Creation of friendship between Alice and Charlie

Instance: Lab_9_PES1UG23CS196 Database: neo4j_cypher_5 User: neo4j

```
1 MATCH (alice:Student {name: "Alice"}),
2     (charlie:Student {name: "Charlie"})
3 CREATE (alice)-[:FRIEND_WITH]-(charlie),
4     (charlie)-[:FRIEND_WITH]-(alice)
5 RETURN alice, charlie;
6
```



Overview

Node labels

- * (2)
- Student (2)

Relationship types

- * (2)
- FRIEND_WITH (2)

Displaying 2 nodes, 0 relationships.

Part C: Query the Graph

7. Listing All Students

neo4j\$ MATCH (s:Student) RETURN s.name AS Name, s.age AS Age, s.major AS Major;

	Name	Age	Major
1	"Alice"	21	"CSE"
2	"Bob"	22	"ECE"
3	"Charlie"	20	"CSE"

Started streaming 3 records after 7 ms and completed after 8 ms.

8. Finding Courses Taught by Dr. Smith

Instance: Lab_9_PES1UG23CS196 Database: neo4j CYPHER 5 User: neo4j

```
neo4j$ MATCH (:Professor {name: "Dr. Smith"})-[:TEACHES]->(c:Course) RETURN c.code AS CourseCod...
```

	CourseCode	CourseTitle
1	"CS101"	"DATA STRUCTURES"

Started streaming 1 records after 10 ms and completed after 14 ms.

9. Finding Friends of Charlie

Instance: Lab_9_PES1UG23CS196 Database: neo4j CYPHER 5 User: neo4j

```
1 MATCH (charlie:Student {name: "Charlie"})<-[:FRIEND_WITH]-(friend:Student)
2 RETURN friend.name AS FriendName;
3
```

	FriendName
1	"Alice"

Started streaming 1 records after 10 ms and completed after 11 ms.

10. Listing All Students in the Same Course

Instance: Lab_9_PES1UG23CS196 Database: neo4j CYPHER 5 User: neo4j

```
1 MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
2 RETURN c.code AS CourseCode, c.title AS CourseTitle, collect(s.name) AS Students;
3
```

	CourseCode	CourseTitle	Students
1	"CS101"	"DATA STRUCTURES"	["Alice"]
2	"EC202"	"DIGITAL SYSTEMS"	["Bob"]

Started streaming 2 records after 18 ms and completed after 20 ms.

11. Finding Professors Who Teach Alice's Courses

Instance: Lab_9_PES1UG23CS196 Database: neo4j CYPHER 5 User: neo4j

```
1 MATCH (alice:Student {name: "Alice"})-[:ENROLLED_IN]->(c:Course)-[:TEACHES]-
  (prof:Professor)
2 RETURN prof.name AS ProfessorName, c.code AS CourseCode, c.title AS CourseTitle;
3
```

	ProfessorName	CourseCode	CourseTitle
1	"Dr. Smith"	"CS101"	"DATA STRUCTURES"

Started streaming 1 records after 13 ms and completed after 14 ms.

12. Finding Students Who Are Friends and Enrolled in the Same Course

Instance: Lab_9_PES1UG23CS196 Database: neo4j CYPHER 5 User: neo4j

```
1 MATCH (s1:Student)-[:FRIEND_WITH]-(s2:Student),
2       (s1)-[:ENROLLED_IN]->(c:Course)-[:ENROLLED_IN]-(s2)
3 WHERE s1.name < s2.name
4 RETURN s1.name AS Student1, s2.name AS Student2, collect(DISTINCT c.code) AS SharedCourses;
5
```

(no changes, no records)

Completed after 21 ms.

13. Finding Courses with More Than One Student Enrolled

Instance: Lab_9_PES1UG23CS196 Database: neo4j CYPHER 5 User: neo4j

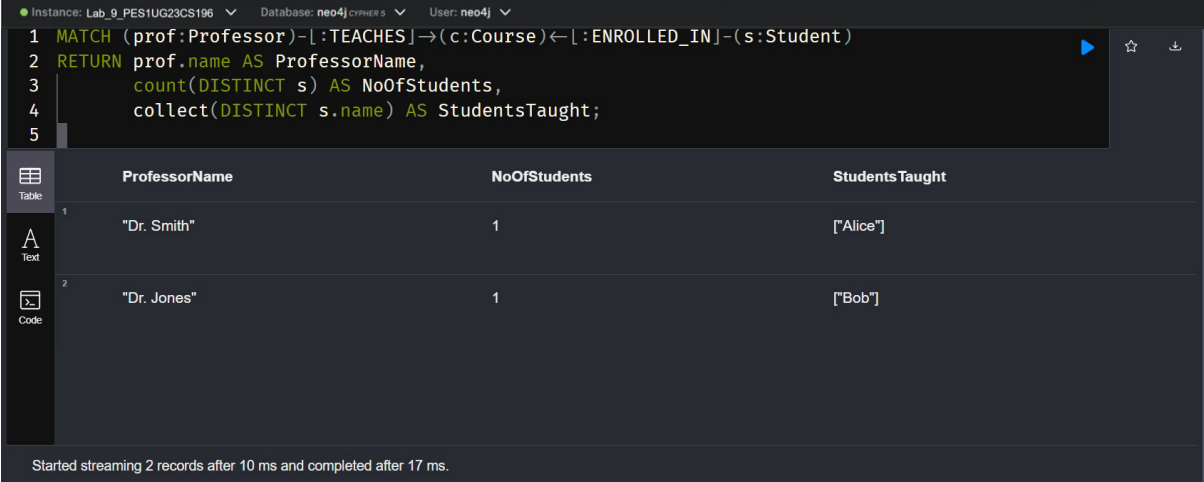
```
neo4j$ MATCH (s:Student)-[:ENROLLED_IN]->(c:Course) WITH c, count(s) AS studentCount, collect(s.name) AS studentNames
```

(no changes, no records)

Completed after 15 ms.

Bonus Query (Advanced)

14. Counting how many students each professor teaches.



The image shows the Neo4j Cypher query interface. At the top, the instance is 'Lab_9_PES1UG23CS196', the database is 'neo4j', and the user is 'neo4j'. The query is as follows:

```
1 MATCH (prof:Professor)-[:TEACHES]-(c:Course)-[:ENROLLED_IN]-(s:Student)
2 RETURN prof.name AS ProfessorName,
3        count(DISTINCT s) AS NoOfStudents,
4        collect(DISTINCT s.name) AS StudentsTaught;
5
```

The results are displayed in a table with three columns: ProfessorName, NoOfStudents, and Students Taught. There are two records:

	ProfessorName	NoOfStudents	Students Taught
1	"Dr. Smith"	1	["Alice"]
2	"Dr. Jones"	1	["Bob"]

At the bottom, a status message reads: 'Started streaming 2 records after 10 ms and completed after 17 ms.'